LABORATORY PROJECT REPORT

BLUETOOTH DATA INTERFACING

EXPERIMENT 9

Section 1

Group 2

Semester 2 2025/2026

NO	NAME	MATRIC NO
1	AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	2310663
2	ALYA SYAFINA BINTI ABDUL RAZAK	2218084

DATE OF SUBMISSION: 13 MAY 2026

Abstract

This experiment focuses on developing a wireless monitoring and control system using Bluetooth communication between an ESP32 or an Arduino equipped with an HC-05 module and an external device such as a smartphone or computer. Instead of using a temperature sensor, a potentiometer is used to generate variable analog input values, which are then transmitted through Bluetooth for real-time monitoring. In addition, simple control commands such as “FAN ON” and “FAN OFF” are included to demonstrate remote control functionality. Overall, the experiment demonstrates how Bluetooth Serial communication can be used to establish two-way interaction between a microcontroller and a connected device for IoT-based monitoring and control applications.

Table of Contents

1.0 Introduction

Wireless communication has become an important part of modern embedded systems, especially in IoT (Internet of Things) and remote monitoring applications. Among the available wireless technologies, Bluetooth is widely used because it is easy to implement, consumes low power, and can easily connect with smartphones and computers.

This project develops a Bluetooth-based monitoring and control system using an ESP32 or Arduino together with an HC-05 Bluetooth module. Instead of using a temperature sensor, a potentiometer is used as the input device to provide variable analog readings to the microcontroller. The potentiometer values are then transmitted wirelessly through Bluetooth to a connected smartphone or computer for real-time monitoring. At the same time, the system can receive user commands to control external devices such as a fan or LED indicator, demonstrating simple remote-control functionality.

The main objective of this project is to understand analog data acquisition, Bluetooth serial communication, and two-way communication between a microcontroller and an external device. This project also introduces concepts commonly applied in IoT systems, wireless monitoring, smart home automation, and remote control applications.

2.0 Materials and Equipment

2.1 Electronic Components

- ESP32 dev board
- Potentiometer
- Smartphone or computer with Bluetooth support

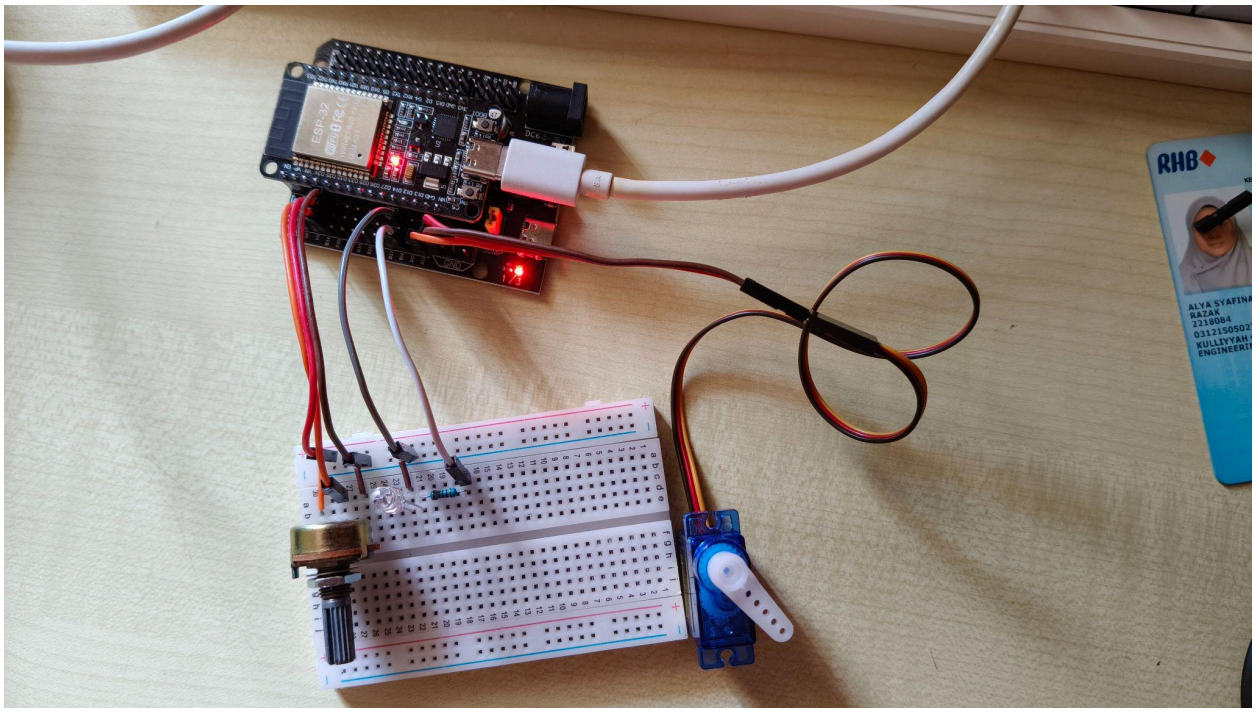
2.2 Equipment and Tools

- Power supply for the ESP32

- Breadboard
- Jumper wires

3.0 Experimental Setup

1. Hardware Setup:
   - Connect the potetntiometer sensor to the ESP32.
   - Power up the system.
2. Arduino Programming:
   Write an Arduino sketch to:
   - Read temperature data from the potetntiometer sensor.
   - Transmit the data over Bluetooth serial connection.
   - Receive simple commands (e.g., ” ON”, “ OFF”) via Bluetooth.
3. Bluetooth Programming:
   - Pair the ESP32 Bluetooth with the smartphone.
   - Use a serial terminal app to view data and send commands.
4. Remote Monitoring:
   - Observe real-time temperature readings on paired device.
   - Send control commands to toggle the LED.



4.0 Methodology

Hardware Setup

- ESP32 development board **or** Arduino Uno with HC-05 Bluetooth module
- DHT22 temperature and humidity sensor connected to the microcontroller
- Jumper wires and breadboard
- Power supply via USB
- Smartphone/PC with Bluetooth terminal app

Steps:

1. Connect the potentiometer terminals properly: one side terminal to 3.3V/5V, the other side terminal to GND, and the middle pin (wiper) to an analog input pin (e.g., A0 for Arduino or VP/GPIO36 for ESP32).
2. For Arduino: Connect HC-05 TX → Arduino RX and HC-05 RX → Arduino TX using SoftwareSerial pins 2 and 3.
3. Power up the microcontroller using a USB connection.
4. Ensure all ground (GND) connections are shared between the microcontroller, HC-05 module, and potentiometer.

Programming codes

```
#include "BluetoothSerial.h"

#include <ESP32Servo.h>

BluetoothSerial SerialBT;

Servo myServo;

// Pin Definitions
```

```
const int potpin = 34; // Potentiometer input

const int servoPin = 13; // Signal pin for the Servo

const int LED = 14; // Built-in LED for feedback

char command;

int potValue = 0;

void setup() {

Serial.begin(9600);

// Start Bluetooth with your preferred display name

SerialBT.begin("Group 2");

// Setup hardware

myServo.attach(servoPin);

pinMode(LED, OUTPUT);

Serial.println("System Initialized. Connect your smartphone via Bluetooth.");

}

void loop() {

// 1. Read Potentiometer (Sensor) data

potValue = analogRead(potpin);

// 2. Transmit the value to the smartphone terminal

SerialBT.print("Sensor (Pot) Value: ");
```

```
SerialBT.println(potValue);

// 3. Check for incoming Bluetooth commands

if (SerialBT.available()) {

command = SerialBT.read();

// Process commands to move the Servo

switch (command) {

case '1': // Move to start position

myServo.write(0);

digitalWrite(LED, HIGH);

SerialBT.println("Action: Servo set to 0°");

break;

case '2': // Move to middle position

myServo.write(90);

SerialBT.println("Action: Servo set to 90°");

break;

case '3': // Move to end position

myServo.write(180);

digitalWrite(LED, LOW);

SerialBT.println("Action: Servo set to 180°");
```

```
break;
}
}
// Delay to prevent flooding the Bluetooth terminal
delay(1000);
}
```

5.0 Data Collection

The data collection process was carried out using an ESP32 microcontroller interfaced with a potentiometer as the analog input device. The potentiometer was connected to an analog input pin (e.g., GPIO 34), where it provided variable voltage readings based on the rotation of its knob. Bluetooth communication was established using the ESP32's built-in Bluetooth module. The system was powered via USB, and the Bluetooth connection was paired with a smartphone using a serial Bluetooth terminal application.

The analog values from the potentiometer were transmitted wirelessly from the ESP32 to the paired device at intervals of approximately 2 seconds. The received data were displayed in real time on the Bluetooth terminal application. During the experiment, the values were continuously observed and manually recorded over several minutes to monitor changes in the analog input.

In addition to data transmission, remote control commands were tested by sending simple text instructions such as "ON" and " OFF" from the Bluetooth terminal. These commands were used to toggle the onboard LED of the ESP32, which acted as a simulated control or indicator output. The corresponding responses were observed to confirm successful two-way Bluetooth communication. All potentiometer readings and system responses were documented for further analysis.

6.0 Data Analysis

The data collected from the potentiometer was analyzed based on the changes in its analog output as detected by the ESP32 microcontroller. As the potentiometer knob was adjusted, it produced varying voltage levels, which were converted into digital values using the microcontroller's ADC. These values were then transmitted to a smartphone via Bluetooth at intervals of approximately 2 seconds for real-time monitoring.

The results showed that the potentiometer readings changed smoothly and proportionally with the rotation of the knob. When turned toward one end, the values decreased toward the lower ADC range, while turning it in the opposite direction increased the values toward the maximum range. Some minor fluctuations were observed during data collection, which may be due to electrical noise or slight instability in the analog signal. However, these variations did not significantly affect the overall accuracy of the readings.

In addition, the Bluetooth communication remained stable throughout the experiment, with consistent data transmission and no noticeable delays. The system also responded correctly to control commands such as "ON" and "OFF," confirming successful two-way communication between the microcontroller and the external device. Overall, the system effectively demonstrated reliable real-time analog data monitoring using a potentiometer in an IoT-based environment.

7.0 Results

The ESP32 and the connected device successfully communicated via Bluetooth during the experiment. Instead of a temperature sensor, a potentiometer was used as the input device,

generating variable analog values based on knob rotation. These readings were transmitted every two seconds and were reliably displayed on the Bluetooth terminal application. The data showed smooth and consistent changes in value corresponding to adjustments of the potentiometer, confirming that both the sensor input and communication system were functioning correctly. The transmission remained stable throughout the experiment with no noticeable delays, except for occasional brief fluctuations that may be due to minor ADC noise. In addition to data transmission, the system also supported remote control functionality. Commands such as “ON” and “OFF” were sent through the Bluetooth terminal and were immediately received by the microcontroller. In response, the ESP32 successfully toggled the simulated output, demonstrating proper execution of control instructions without delay. Overall, the results confirmed that the Bluetooth connection effectively enabled both real-time monitoring of potentiometer values and reliable two-way communication for remote control purposes.

8.0 Discussion

The demonstration effectively illustrated how Bluetooth technology can be used for wireless communication in embedded systems. In this setup, a potentiometer was used instead of a temperature and humidity sensor to provide variable analog input values. These values were transmitted consistently to the connected device using the HC-05/ESP32 Bluetooth serial module. The stable Bluetooth connection during the monitoring process highlighted its suitability for low-power IoT applications.

When control commands were sent from a smartphone or computer, the microcontroller responded successfully, demonstrating proper handling of serial input. This functionality simulates basic remote control of external devices such as lights, fans, or other actuators. The fast response also indicated that Bluetooth communication provides low latency for simple control applications.

The 2-second sampling interval introduced slight delays between data updates, but this was appropriate for observing gradual changes in potentiometer readings. Unlike environmental sensors, the potentiometer provides immediate and continuous variation, making it suitable for real-time input demonstration. Overall, the system successfully demonstrates wireless data transmission, microcontroller programming, and basic IoT-based remote control using simple serial communication.

9.0 Conclusion

The objective of the experiment, which was to develop a Bluetooth-based monitoring and control system, was successfully achieved. Instead of a temperature sensor, a potentiometer was used to provide variable analog input values, which were reliably read by the ESP32/Arduino and transmitted to a PC or smartphone. The system consistently displayed the potentiometer readings in real time, demonstrating stable and accurate data communication. In addition to monitoring, the Bluetooth connection also supported user-defined commands to control external outputs. The microcontroller responded effectively to instructions sent from the connected device, showing proper execution of serial communication for basic control functions. This confirmed that the system could handle both data transmission and command reception without significant delay. Overall, the experiment demonstrates the core principles of IoT connectivity, including wireless data transfer, real-time monitoring, and remote control using Bluetooth communication. It also provides a strong foundation for developing more advanced embedded systems and smart device applications in the future.

10.0 Recommendations

A number of modifications might be implemented in the future to boost the system's utility, accuracy, and efficiency. First off, employing a more sophisticated sensor, like the BME280 or DS18B20, would increase measurement accuracy and yield more consistent readings. Long-term monitoring and trend analysis would be possible with the implementation of a data recording capability, either via an SD card module or by uploading values to a cloud platform. The simple Bluetooth terminal should be replaced with a specialised desktop or mobile interface that provides greater visualisation through graphs and real-time dashboards. Multiple actuators may be managed with additional control functions, increasing the system's adaptability for automation or smart home applications. Last but not least, enhancing communication security, for example, by using encrypted Bluetooth protocols or password protection, would make the system more appropriate for actual IoT deployments where data security and integrity are crucial.

11.0 References

- **Arduino Bluetooth Basic Tutorial**
- **Arduino and HC-05 Bluetooth Module Complete Tutorial**
- **Basic Bluetooth communication with Arduino & HC-05**

12. Acknowledgements

Special thanks to Dr. ZULKIFLI BIN ZAINAL ABIDIN and Dr. WAHJU SEDIONO for their guidance and support during this experiment.

Student's Declaration

Certificate of Originality and Authenticity

We hereby certify that we are collectively responsible for the work presented in this report. The content reflects our original work, except where specific references or acknowledgements have been made. No part of this report has been completed or submitted by individuals or sources not identified within.

Furthermore, we confirm that this report is the result of a collaborative effort, with contributions made by all group members. The extent of each individual's contribution is documented within this certificate.

We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We therefore, agreed unanimously that this report shall be submitted for marking and this final printed report has been verified by us.

Name: AHMAD SYAHMI WAJDI BIN AHMAD SHUKRI	Read	[/]
Matric Number: 2310663	Understand	[/]
Signature: *SYAHMI*	Agree	[/]

Name: ALYA SYAFINA BINTI ABDUL RAZAK	Read	[/]
Matric Number: 2218084	Understand	[/]
Signature: *SYAF*	Agree	[/]